Homework 2

Done by: Vladyslav Danylyshyn and Iia Maharyta (team: Miguel)

```
!pip install -q torch torchvision --index-url https://download.pytorch.org/whl/cu128
!pip install -q --upgrade-strategy only-if-needed pycocotools numpy pandas scikit-learn opencv-python pyyaml matplotli

import torch

print("torch", torch.__version__, "cuda build", torch.version.cuda)

if not torch.cuda.is_available():
    raise RuntimeError(
        "CUDA is not available. Restart the kernel and run this cell again. "
        "If it still fails, the driver is older than the cu128 wheel: change the index URL to cu124."
    )

name = torch.cuda.get_device_name(0)
vram = torch.cuda.get_device_properties(0).total_memory / 1024**3
print(f"GPU {name}  {vram:.1f} GB")
print("packages ok")
```

```
torch 2.15.0+cu132 cuda build 13.2
GPU NVIDIA GeForce RTX 4090 Laptop GPU  15.6 GB
packages ok
```

```
import json
import numpy as np
import pandas as pd
import cv2
import yaml
import matplotlib.pyplot as plt
from pathlib import Path
from PIL import Image

from sklearn.preprocessing import MinMaxScaler
from sklearn.utils import check_random_state

IMGSZ = 1280
PAD_VALUE = 114
FRAME_STRIDE = 5  # measured in the lag cell; the loader keeps every frame
SEED = 0

cv2.setNumThreads(0)
```

```
!pip install -q pycocotools
from pycocotools.coco import COCO
```

Data Understanding

```
import os
from google.colab import userdata

os.environ["KAGGLE_API_TOKEN"] = userdata.get("kaggle").strip()
```

```
!kaggle competitions download -c multiple-object-video-detection-hw-2-ucu-ai-2026 -p data
!unzip -q -o data/*.zip -d data
```

```
Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to the la
Downloading multiple-object-video-detection-hw-2-ucu-ai-2026.zip to data
100% 9.88G/9.88G [02:18<00:00, 76.8MB/s]
```

```
ROOT = Path("data")
RESULTS = Path("results")
BOX_TOL = 1e-3
```

```
def class_names(split):
    meta = yaml.safe_load((ROOT / split / "data.yaml").read_text())
    names = meta["names"]
    if isinstance(names, dict):
        return {int(k): v for k, v in names.items()}
    return dict(enumerate(names))


def load_split(split):
    # short_line is skipped. The other counters stay in the table.
    split_dir = ROOT / split
    rows = []
    issues = {"short_line": 0, "nonpositive": 0, "out_of_range": 0, "bad_class": 0}
    for seq_dir in sorted((split_dir / "images").iterdir()):
        if not seq_dir.is_dir():
            continue
        seq = seq_dir.name
        for img_path in sorted(seq_dir.glob("*.jpg")):
            stem = img_path.stem
            label_path = split_dir / "labels" / seq / f"{stem}.txt"
            ignore_path = split_dir / "ignore" / seq / f"{stem}.txt"
            pairs = [("object", label_path), ("ignore", ignore_path)]
            any_box = False
            for kind, txt_path in pairs:
                if not txt_path.exists():
                    continue
                for line in txt_path.read_text().splitlines():
                    vals = line.split()
                    if not vals:
                        continue
                    if len(vals) < 4:
                        issues["short_line"] += 1
                        continue
                    if len(vals) == 4:
                        cls_id, cx, cy, bw, bh = None, *map(float, vals)
                    else:
                        cls_id = int(float(vals[0]))
                        cx, cy, bw, bh = map(float, vals[1:5])
                    if bw <= 0 or bh <= 0:
                        issues["nonpositive"] += 1
                    x1, y1 = cx - bw / 2, cy - bh / 2
                    x2, y2 = cx + bw / 2, cy + bh / 2
                    if (
                        min(x1, y1, cx, cy, bw, bh) < -BOX_TOL
                        or max(x2, y2, cx, cy, bw, bh) > 1 + BOX_TOL
                    ):
                        issues["out_of_range"] += 1
                    if kind == "object" and cls_id not in names:
                        issues["bad_class"] += 1
                    rows.append({
                        "split": split,
                        "sequence": seq,
                        "frame": stem,
                        "image_path": str(img_path),
                        "kind": kind,
                        "class_id": cls_id,
                        "cx": cx, "cy": cy, "w": bw, "h": bh,
                    })
                    any_box = True
            if not any_box:
                rows.append({
                    "split": split,
                    "sequence": seq,
                    "frame": stem,
                    "image_path": str(img_path),
                    "kind": "empty",
                    "class_id": None,
                    "cx": None, "cy": None, "w": None, "h": None,
                })
    return pd.DataFrame(rows), issues


names = class_names("train")
train_df, train_issues = load_split("train")
val_df, val_issues = load_split("val")
```

```
test_df = pd.read_csv(ROOT / "test_images.csv")
test_df["image_path"] = test_df["file_name"].map(lambda p: str(ROOT / p))

print("classes:", names)
print(
    "train", train_df["sequence"].nunique(), "videos,",
    train_df["image_path"].nunique(), "frames,", len(train_df), "rows",
)
print(
    "val  ", val_df["sequence"].nunique(), "videos,",
    val_df["image_path"].nunique(), "frames,", len(val_df), "rows",
)
print("test ", test_df["clip"].nunique(), "videos,", len(test_df), "frames")
print("label issues: short_line is skipped; the other rows are stored")
print("train", train_issues)
print("val", val_issues)
train_df.head()
```

```
classes: {0: 'person', 1: 'bicycle', 2: 'car', 3: 'van', 4: 'truck', 5: 'tricycle', 6: 'bus', 7: 'motor'}
train 56 videos, 24201 frames, 1169362 rows
val   7 videos, 2846 frames, 118127 rows
test  48 videos, 3811 frames
label issues: short_line is skipped; the other rows are stored
train {'short_line': 0, 'nonpositive': 0, 'out_of_range': 0, 'bad_class': 0}
val {'short_line': 0, 'nonpositive': 0, 'out_of_range': 0, 'bad_class': 0}
```

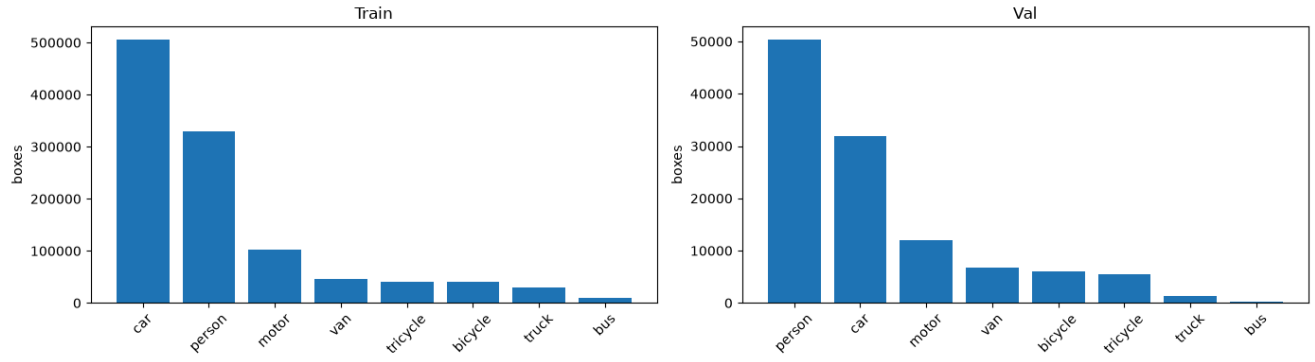
	split	sequence	frame	image_path	kind	class_id	cx	cy	w
0	train	uav0000013_00000_v	0000001	data/train/images/uav0000013_00000_v/0000001.jpg	object	0.0	0.328497	0.776455	0.026042
1	train	uav0000013_00000_v	0000001	data/train/images/uav0000013_00000_v/0000001.jpg	object	0.0	0.322917	0.757275	0.026786
2	train	uav0000013_00000_v	0000001	data/train/images/uav0000013_00000_v/0000001.jpg	object	5.0	0.427455	0.923280	0.064732
3	train	uav0000013_00000_v	0000001	data/train/images/uav0000013_00000_v/0000001.jpg	object	0.0	0.451637	0.610450	0.023810

```
def class_counts(df):
    objects = df[df["kind"] == "object"].copy()
    objects["class_name"] = objects["class_id"].map(names)
    counts = (
        objects["class_name"]
        .value_counts()
        .rename_axis("class_name")
        .reset_index(name="boxes")
    )
    counts["share"] = counts["boxes"] / counts["boxes"].sum()
    return objects, counts

train_obj, train_class_counts = class_counts(train_df)
val_obj, val_class_counts = class_counts(val_df)
print("TRAIN")
print(train_class_counts.to_string(index=False))
print("\nVAL")
print(val_class_counts.to_string(index=False))
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
for ax, counts, title in (
    (axes[0], train_class_counts, "Train"),
    (axes[1], val_class_counts, "Val"),
):
    ax.bar(counts["class_name"], counts["boxes"])
    ax.set_title(title)
    ax.tick_params(axis="x", rotation=45)
    ax.set_ylabel("boxes")
fig.tight_layout()
plt.show()
```

```
TRAIN
class_name  boxes    share
       car 505301 0.457073
    person 328701 0.297328
     motor 102819 0.093005
       van  46940 0.042460
  tricycle  41349 0.037402
   bicycle  40255 0.036413
     truck  30498 0.027587
       bus   9653 0.008732

VAL
class_name  boxes    share
    person  50312 0.440823
       car  31821 0.278809
     motor  12025 0.105360
       van   6842 0.059948
   bicycle   6022 0.052763
  tricycle   5487 0.048076
     truck   1359 0.011907
       bus    264 0.002313
```



```
per_seq = (
    train_obj.groupby(["sequence", "class_name"])
    .size()
    .unstack(fill_value=0)
)
per_seq_share = per_seq.div(per_seq.sum(axis=1), axis=0)

fig, ax = plt.subplots(figsize=(14, 5))
per_seq_share.plot(kind="bar", stacked=True, ax=ax, width=0.9)
ax.set_title("Train class mix by sequence")
ax.set_ylabel("share of boxes")
ax.legend(loc="center left", bbox_to_anchor=(1, 0.5))
ax.set_xticklabels([])
fig.tight_layout()
plt.show()

dominant = pd.DataFrame({
    "class_name": per_seq_share.idxmax(axis=1),
    "share": per_seq_share.max(axis=1),
}).sort_values("share", ascending=False)
print(
    "dominant-class share within a sequence: "
    f"min {dominant['share'].min():.1%}, max {dominant['share'].max():.1%}"
)
print("highest share")
display(dominant.head(8))
print("lowest share")
display(dominant.tail(5))
pure = dominant[dominant["share"] >= 1 - 1e-9]
print(f"sequences with a single class: {len(pure)}")
display(pure)
object_sequences = set(train_df.loc[train_df["kind"] == "object", "sequence"])
named_sequences = set(per_seq_share.index)
print("sequences with no object boxes:", sorted(set(train_df["sequence"]) - object_sequences))
print("sequences whose object rows have no known class:", sorted(object_sequences - named_sequences))
```




```
dominant-class share within a sequence: min 34.6%, max 100.0%
highest share
```

sequence	class_name	share
uav0000099_02109_v	person	1.000000

```
def read_size(path):
    with Image.open(path) as im:
        return im.size


def frame_sizes(df):
    frames = (
        df.drop_duplicates(["sequence", "frame"])
        [["sequence", "frame", "image_path"]]
        .reset_index(drop=True)
    )
    wh = [read_size(path) for path in frames["image_path"]]
    frames["img_w"] = [w for w, h in wh]
    frames["img_h"] = [h for w, h in wh]
    return frames
```

```
def resolution_report(frames, title):
    mixed = int((
        frames.groupby("sequence")[["img_w", "img_h"]].nunique().max(axis=1) > 1
    ).sum())
    counts = (
        frames.groupby(["img_w", "img_h"])
        .size()
        .rename("frames")
        .reset_index()
        .sort_values("frames", ascending=False)
    )
    print(title)
    print(f"sequences with more than one resolution: {mixed}")
    display(counts)
    return counts, mixed


def with_pixel_size(df, sizes):
    objects = df[df["kind"] == "object"].merge(
        sizes[["sequence", "frame", "img_w", "img_h"]],
        on=["sequence", "frame"],
        how="left",
    )
    objects["class_name"] = objects["class_id"].map(names)
    objects["box_w"] = objects["w"] * objects["img_w"]
    objects["box_h"] = objects["h"] * objects["img_h"]
    objects["area"] = objects["box_w"] * objects["box_h"]
    return objects


def per_frame_counts(df):
    frames = df.drop_duplicates(["sequence", "frame"])[["sequence", "frame"]]
    counts = (
        df[df["kind"] == "object"]
        .groupby(["sequence", "frame"])
        .size()
        .rename("n_objects")
        .reset_index()
    )
    return frames.merge(counts, on=["sequence", "frame"], how="left").fillna({"n_objects": 0})


train_frame_sizes = frame_sizes(train_df)
val_frame_sizes = frame_sizes(val_df)
resolution_report(train_frame_sizes, "TRAIN resolutions")
resolution_report(val_frame_sizes, "VAL resolutions")
train_boxes = with_pixel_size(train_df, train_frame_sizes)
val_boxes = with_pixel_size(val_df, val_frame_sizes)
train_frame_counts = per_frame_counts(train_df)
val_frame_counts = per_frame_counts(val_df)
```

```
TRAIN resolutions
sequences with more than one resolution: 0
```

	img_w	img_h	frames
2	1904	1071	12895
0	1344	756	5645
4	2688	1512	3505
1	1360	765	962
3	1920	1080	721
5	2720	1530	473

```
VAL resolutions
sequences with more than one resolution: 0
```

	img_w	img_h	frames
4	3840	2160	978
0	1344	756	827
1	1904	1071	459
3	2720	1530	349
2	2688	1512	233

```
def print_box_size(width, height, area, title):
    print(title)
    print(
        f"width  px: median {width.median():.0f}, "
        f"p10 {width.quantile(.1):.0f}, p90 {width.quantile(.9):.0f}"
    )
    print(
        f"height px: median {height.median():.0f}, "
        f"p10 {height.quantile(.1):.0f}, p90 {height.quantile(.9):.0f}"
    )
    small = float((area < 32 ** 2).mean())
    medium = float(((area >= 32 ** 2) & (area < 96 ** 2)).mean())
    large = float((area >= 96 ** 2).mean())
    print(f"area bins: small {small:.1%}, medium {medium:.1%}, large {large:.1%}")
    return small, medium, large


def summarize(boxes, counts, title):
    print_box_size(boxes["box_w"], boxes["box_h"], boxes["area"], f"\n{title} native pixels")
    print(
        f"objects/frame: median {counts.n_objects.median():.0f}, "
        f"mean {counts.n_objects.mean():.1f}, max {counts.n_objects.max():.0f}"
    )


summarize(train_boxes, train_frame_counts, "TRAIN")
summarize(val_boxes, val_frame_counts, "VAL")
peak = train_frame_counts.loc[train_frame_counts["n_objects"].idxmax()]
print(
    "train frame with the most objects: "
    f"{peak.sequence} {peak.frame} n={peak.n_objects:.0f}"
)

by_seq = train_frame_counts.groupby("sequence")["n_objects"].agg(["median", "mean", "max"])
by_seq["median_area"] = train_boxes.groupby("sequence")["area"].median()
by_seq = by_seq.dropna(subset=["median_area"]).sort_values("mean", ascending=False)
corr = by_seq["mean"].corr(by_seq["median_area"], method="spearman")
print(f"spearman mean objects/frame vs median box area: {corr:.3f}")
print("densest sequences")
display(by_seq.head(5))
print("sparsest sequences")
display(by_seq.tail(5))

aspect = train_boxes["box_w"] / train_boxes["box_h"].clip(lower=1)
print(
    "native aspect w/h p10, p50, p90:",
    f"{aspect.quantile(0.1):.2f}",
    f"{aspect.median():.2f}",
    f"{aspect.quantile(0.9):.2f}",
)
```

```
test_per_clip = test_df.groupby("clip").size()
test_wh = [read_size(path) for path in test_df["image_path"]]
test_sizes = test_df[["clip"]].copy()
test_sizes["img_w"] = [w for w, h in test_wh]
test_sizes["img_h"] = [h for w, h in test_wh]
print("\nTEST")
print(f"clips {test_df['clip'].nunique()}, frames {len(test_df)}")
print(
    f"frames/clip: median {test_per_clip.median():.0f}, "
    f"mean {test_per_clip.mean():.1f}, max {test_per_clip.max():.0f}"
)
print("frames at each resolution")
display(
    test_sizes.groupby(["img_w", "img_h"])
    .size()
    .rename("frames")
    .reset_index()
    .sort_values("frames", ascending=False)
)
print("no boxes: test labels are not provided")

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
axes[0].hist(np.sqrt(train_boxes["area"].clip(lower=0)), bins=50)
axes[0].set_title("Train object size, native sqrt(area)")
axes[0].set_xlabel("sqrt(width * height)")
axes[1].hist(train_frame_counts["n_objects"], bins=40)
axes[1].set_title("Train objects per frame")
axes[1].set_xlabel("objects")
axes[2].hist(test_per_clip, bins=20)
axes[2].set_title("Test frames per clip")
axes[2].set_xlabel("frames")
fig.tight_layout()
plt.show()
```

```
TRAIN native pixels
width  px: median 44, p10 19, p90 126
height px: median 48, p10 25, p90 97
area bins: small 24.9%, medium 63.5%, large 11.6%
objects/frame: median 43, mean 45.7, max 180

VAL native pixels
width  px: median 47, p10 20, p90 110
height px: median 57, p10 29, p90 116
area bins: small 19.2%, medium 69.5%, large 11.3%
objects/frame: median 37, mean 40.1, max 116
train frame with the most objects: uav0000295_02300_v 0000322 n=180
spearman mean objects/frame vs median box area: -0.008
densest sequences
```

sequence	median	mean	max	median_area
uav0000138_00000_v	150.0	142.962441	156.0	2059.073456
uav0000126_00001_v	130.0	132.218447	166.0	2340.021696
uav0000124_00944_v	129.0	124.567839	134.0	2145.001127
uav0000072_04488_v	108.0	110.376471	124.0	2099.944350
uav0000150_02310_v	102.0	108.885057	174.0	3603.988592

```
sparsest sequences
```

sequence	median	mean	max	median_area
uav0000099_02109_v	7.0	7.135458	13.0	1374.979820
uav0000266_03598_v	6.0	6.474453	10.0	6548.412416
uav0000263_03289_v	7.0	6.463448	11.0	3008.958639
uav0000266_04830_v	6.0	6.086207	7.0	6655.911424
uav0000218_00001_v	5.0	5.146667	7.0	3861.547948

```
native aspect w/h p10, p50, p90: 0.44 0.95 2.13

TEST
clips 48, frames 3811
frames/clip: median 74, mean 79.4, max 148
frames at each resolution
```

	img_w	img_h	frames
3	1360	764	1474
5	1920	1080	991
2	1344	756	441
0	680	382	321
4	1904	1072	305

```
def coverage(df, title):
    frames = df.drop_duplicates(["sequence", "frame"])
    object_keys = set(zip(
        df.loc[df["kind"] == "object", "sequence"],
        df.loc[df["kind"] == "object", "frame"],
    ))
    all_keys = set(zip(frames["sequence"], frames["frame"]))
    ignore = df[df["kind"] == "ignore"]
    ignore_frames = ignore.drop_duplicates(["sequence", "frame"])
    print(title)
    print(f"frames {len(frames)}")
    print(f"frames with no object boxes {len(all_keys - object_keys)}")
    print(f"frames with no boxes at all {int((df['kind'] == 'empty').sum())}")
    print(f"ignore boxes {len(ignore)} on {len(ignore_frames)} frames")


coverage(train_df, "TRAIN coverage")
coverage(val_df, "VAL coverage")
```

```
TRAIN coverage
frames 24201
frames with no object boxes 3
```

```
frames with no boxes at all 3
ignore boxes 63843 on 18090 frames
VAL coverage
frames 2846
frames with no object boxes 0
frames with no boxes at all 0
ignore boxes 3995 on 1434 frames
```

```
def valid_class_id(value):
    if value is None or (isinstance(value, float) and np.isnan(value)):
        return None
    try:
        parsed = int(value)
    except (TypeError, ValueError):
        return None
    if parsed not in names:
        return None
    return parsed


def build_coco(df, sizes, split):
    # area is the original-frame box. Letterbox size is a separate table.
    ignore_id = int(max(names) + 1)
    frames = sizes[["sequence", "frame", "img_w", "img_h"]].reset_index(drop=True)
    images = []
    image_ids = {}
    for image_id, row in enumerate(frames.itertuples(index=False), start=1):
        image_ids[(row.sequence, row.frame)] = image_id
        images.append({
            "id": image_id,
            "file_name": f"{split}/images/{row.sequence}/{row.frame}.jpg",
            "width": int(row.img_w),
            "height": int(row.img_h),
            "sequence": row.sequence,
            "frame": str(row.frame),
        })
    labeled = df[df["kind"].isin(["object", "ignore"])].merge(
        frames, on=["sequence", "frame"], how="left",
    )
    annotations = []
    skipped = 0
    for row in labeled.itertuples(index=False):
        if row.kind == "object":
            category_id = valid_class_id(row.class_id)
            if category_id is None:
                skipped += 1
                continue
            iscrowd = 0
        else:
            category_id = ignore_id
            iscrowd = 1
        bw = float(row.w) * int(row.img_w)
        bh = float(row.h) * int(row.img_h)
        annotations.append({
            "id": len(annotations) + 1,
            "image_id": image_ids[(row.sequence, row.frame)],
            "category_id": category_id,
            "bbox": [
                (float(row.cx) - float(row.w) / 2.0) * int(row.img_w),
                (float(row.cy) - float(row.h) / 2.0) * int(row.img_h),
                bw,
                bh,
            ],
            "area": float(bw * bh),
            "iscrowd": iscrowd,
        })
    categories = [
        {"id": int(class_id), "name": class_name}
        for class_id, class_name in sorted(names.items())
    ]
    categories.append({"id": ignore_id, "name": "ignore"})
    dataset = {
        "info": {"description": f"{split} boxes in original-frame pixels", "year": 2026},
        "licenses": [],
        "images": images,
        "annotations": annotations,
        "categories": categories,
```

```
        }
        path = ROOT / "annotations" / f"instances_{split}.json"
        path.parent.mkdir(parents=True, exist_ok=True)
        with path.open("w") as handle:
            json.dump(dataset, handle, separators=(",", ":"))
        print(
            f"{path} images {len(images)} annotations {len(annotations)} "
            f"skipped_bad_objects {skipped} ignore_category {ignore_id}"
        )
        return path


    def print_coco_bins(coco):
        # pycocotools keeps a box when low < area < high. Bounds are strict.
        object_ids = coco.getAnnIds(iscrowd=0)
        ignore_ids = coco.getAnnIds(iscrowd=1)
        print(f"objects {len(object_ids)}  ignore {len(ignore_ids)}  images {len(coco.imgs)}")
        ranges = (
            ("small", [0, 32 ** 2]),
            ("medium", [32 ** 2, 96 ** 2]),
            ("large", [96 ** 2, 1e5 ** 2]),
        )
        for label, area_rng in ranges:
            n = len(coco.getAnnIds(iscrowd=0, areaRng=area_rng))
            share = n / len(object_ids) if object_ids else float("nan")
            print(f"  {label} {area_rng[0]:.0f} < area < {area_rng[1]:.0f}: {n} ({share:.1%})")
        for cat in coco.dataset["categories"]:
            crowd = 1 if cat["name"] == "ignore" else 0
            n = len(coco.getAnnIds(catIds=[cat["id"]], iscrowd=crowd))
            print(f"  category {cat['id']} {cat['name']}: {n}")


    train_coco_path = build_coco(train_df, train_frame_sizes, "train")
    val_coco_path = build_coco(val_df, val_frame_sizes, "val")
    print("TRAIN coco, native pixels")
    print_coco_bins(COCO(str(train_coco_path)))
    print("VAL coco, native pixels")
    print_coco_bins(COCO(str(val_coco_path)))
```

```
data/annotations/instances_train.json images 24201 annotations 1169359 skipped_bad_objects 0 ignore_category 8
data/annotations/instances_val.json images 2846 annotations 118127 skipped_bad_objects 0 ignore_category 8
TRAIN coco, native pixels
loading annotations into memory...
Done (t=1.45s)
creating index...
index created!
objects 1105516  ignore 63843  images 24201
  small 0 < area < 1024: 275368 (24.9%)
  medium 1024 < area < 9216: 702031 (63.5%)
  large 9216 < area < 10000000000: 128117 (11.6%)
  category 0 person: 328701
  category 1 bicycle: 40255
  category 2 car: 505301
  category 3 van: 46940
  category 4 truck: 30498
  category 5 tricycle: 41349
  category 6 bus: 9653
  category 7 motor: 102819
  category 8 ignore: 63843
VAL coco, native pixels
loading annotations into memory...
Done (t=0.13s)
creating index...
index created!
objects 114132  ignore 3995  images 2846
  small 0 < area < 1024: 21918 (19.2%)
  medium 1024 < area < 9216: 79351 (69.5%)
  large 9216 < area < 10000000000: 12863 (11.3%)
  category 0 person: 50312
  category 1 bicycle: 6022
  category 2 car: 31821
  category 3 van: 6842
  category 4 truck: 1359
  category 5 tricycle: 5487
  category 6 bus: 264
  category 7 motor: 12025
  category 8 ignore: 3995
```

```
def add_letterbox_size(objects, new_shape):
    # Matches letterbox(): box width scales by new_shape / max(img_w, img_h).
    out = objects.copy()
    img_w = out["img_w"].to_numpy(dtype=np.float64)
    img_h = out["img_h"].to_numpy(dtype=np.float64)
    ratio = new_shape / np.maximum(img_w, img_h)
    out["lb_w"] = out["w"].to_numpy(dtype=np.float64) * img_w * ratio
    out["lb_h"] = out["h"].to_numpy(dtype=np.float64) * img_h * ratio
    out["lb_area"] = out["lb_w"] * out["lb_h"]
    return out


def size_by_class(objects):
    aspect = objects["lb_w"] / objects["lb_h"].clip(lower=1e-6)
    return (
        objects.assign(aspect=aspect)
        .groupby("class_name")
        .agg(
            boxes=("aspect", "size"),
            median_w=("lb_w", "median"),
            median_h=("lb_h", "median"),
            median_aspect=("aspect", "median"),
        )
        .sort_values("boxes", ascending=False)
    )


def size_by_resolution(objects):
    return (
        objects.groupby(["img_w", "img_h"])
        .agg(
            boxes=("lb_w", "size"),
            median_w=("lb_w", "median"),
            median_h=("lb_h", "median"),
            small_share=("lb_area", lambda area: float((area < 32 ** 2).mean())),
        )
        .reset_index()
        .sort_values("boxes", ascending=False)
    )


print("Letterbox scales every box by side / native long side.")
print("A side of 640 is half the linear size of a side of 1280.")
print("Area bins here are small < 32^2 <= medium < 96^2 <= large.")
train_at_1280 = add_letterbox_size(train_boxes, IMGSZ)
train_at_640 = add_letterbox_size(train_boxes, 640)
val_at_1280 = add_letterbox_size(val_boxes, IMGSZ)
print_box_size(train_at_1280["lb_w"], train_at_1280["lb_h"], train_at_1280["lb_area"], "\nTRAIN at 1280")
print_box_size(train_at_640["lb_w"], train_at_640["lb_h"], train_at_640["lb_area"], "\nTRAIN at 640")
print_box_size(val_at_1280["lb_w"], val_at_1280["lb_h"], val_at_1280["lb_area"], "\nVAL at 1280")
print("\nTRAIN at 1280 by native resolution")
display(size_by_resolution(train_at_1280))
print("VAL at 1280 by native resolution")
display(size_by_resolution(val_at_1280))
train_by_class = size_by_class(train_at_1280)
val_by_class = size_by_class(val_at_1280)
print("TRAIN median width, height, aspect at 1280")
display(train_by_class)
print("VAL median width, height, aspect at 1280")
display(val_by_class)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(np.sqrt(train_at_1280["lb_area"].clip(lower=0)), bins=50)
axes[0].set_title("Train object size at 1280, sqrt(area)")
axes[0].set_xlabel("sqrt(width * height)")
order = train_by_class.sort_values("median_w")
axes[1].bar(order.index.astype(str), order["median_w"])
axes[1].set_title("Train median width at 1280, by class")
axes[1].tick_params(axis="x", rotation=45)
axes[1].set_ylabel("px")
fig.tight_layout()
plt.show()
```

```
Letterbox scales every box by side / native long side.
A side of 640 is half the linear size of a side of 1280.
Area bins here are small < 32^2 <= medium < 96^2 <= large.

TRAIN at 1280
width  px: median 30, p10 13, p90 79
height px: median 32, p10 16, p90 64
area bins: small 54.2%, medium 42.8%, large 3.0%

TRAIN at 640
width  px: median 15, p10 6, p90 40
height px: median 16, p10 8, p90 32
area bins: small 89.0%, medium 10.9%, large 0.1%

VAL at 1280
width  px: median 28, p10 15, p90 68
height px: median 37, p10 16, p90 74
area bins: small 48.4%, medium 49.7%, large 2.0%

TRAIN at 1280 by native resolution
```

img_w	img_h	boxes	median_w	median_h	small_share

```
def frame_sort_key(stem):
    text = str(stem)
    if text.isdigit():
        return (0, int(text))
    return (1, text)


def pairwise_iou(a, b):
    lt = np.maximum(a[:, None, :2], b[None, :, :2])
    rb = np.minimum(a[:, None, 2:], b[None, :, 2:])
    wh = np.clip(rb - lt, 0, None)
    inter = wh[:, :, 0] * wh[:, :, 1]
    area_a = np.clip(a[:, 2] - a[:, 0], 0, None) * np.clip(a[:, 3] - a[:, 1], 0, None)
    area_b = np.clip(b[:, 2] - b[:, 0], 0, None) * np.clip(b[:, 3] - b[:, 1], 0, None)
    union = area_a[:, None] + area_b[None, :] - inter
    return np.where(union > 0, inter / union, 0.0)


def match_fraction(a, b, thr=0.5):
    # Share of the larger set that has a greedy match at IoU >= thr.
    if len(a) == 0 and len(b) == 0:
        return None
    if len(a) == 0 or len(b) == 0:
        return 0.0
    iou = pairwise_iou(a, b)
    matched = 0
    while True:
        index = np.unravel_index(np.argmax(iou), iou.shape)
        if iou[index] < thr:
            break
        matched += 1
        iou[index[0], :] = -1
        iou[:, index[1]] = -1
    return matched / max(len(a), len(b))


def lag_report(df, lags, title):
    print(title)
    print(f"FRAME_STRIDE={FRAME_STRIDE} is not applied. Pairing stored frames.")
    objects = df.loc[df["kind"] == "object", ["sequence", "frame", "cx", "cy", "w", "h"]]
    grouped = {}
    for (seq, frame), group in objects.groupby(["sequence", "frame"], sort=False):
        cx = group["cx"].to_numpy(dtype=np.float64)
        cy = group["cy"].to_numpy(dtype=np.float64)
        w = group["w"].to_numpy(dtype=np.float64)
        h = group["h"].to_numpy(dtype=np.float64)
        grouped[(seq, frame)] = np.stack(
            [cx - w / 2, cy - h / 2, cx + w / 2, cy + h / 2], axis=1,
        )
    frames = df.drop_duplicates(["sequence", "frame"])[["sequence", "frame"]]
    for lag in lags:
        scores = []
        both_empty = 0
        for seq, group in frames.groupby("sequence", sort=False):
            ordered = sorted(group["frame"].tolist(), key=frame_sort_key)
            for i in range(len(ordered) - lag):
                a = grouped.get((seq, ordered[i]))
```

```
                b = grouped.get((seq, ordered[i + lag]))
                if a is None and b is None:
                    both_empty += 1
                    continue
                if a is None:
                    a = np.zeros((0, 4))
                if b is None:
                    b = np.zeros((0, 4))
                scores.append(match_fraction(a, b))
        arr = np.asarray(scores, dtype=np.float64)
        if len(arr) == 0:
            print(f"lag {lag}: pairs 0, both empty skipped {both_empty}")
            continue
        print(
            f"lag {lag}: pairs {len(arr)}, both empty skipped {both_empty}, "
            f"matched at IoU>=0.5 median {np.median(arr):.3f}, mean {arr.mean():.3f}"
        )


lag_report(train_df, (1, FRAME_STRIDE), "TRAIN frame agreement")
lag_report(val_df, (1, FRAME_STRIDE), "VAL frame agreement")
```

```
TRAIN frame agreement
FRAME_STRIDE=5 is not applied. Pairing stored frames.
lag 1: pairs 24143, both empty skipped 2, matched at IoU>=0.5 median 0.990, mean 0.943
lag 5: pairs 23921, both empty skipped 0, matched at IoU>=0.5 median 0.597, mean 0.551
VAL frame agreement
FRAME_STRIDE=5 is not applied. Pairing stored frames.
lag 1: pairs 2839, both empty skipped 0, matched at IoU>=0.5 median 0.929, mean 0.865
lag 5: pairs 2811, both empty skipped 0, matched at IoU>=0.5 median 0.600, mean 0.540
```

Class mix, object size, and frame density belong to a whole video, not to a single frame. Figures quoted here were printed by the previous run of the class table and the native-size summary. Every other result is printed by the cells above and is not copied into this text.

Classes. Train boxes are 45.7% car, 29.7% person, 9.3% motor, then van 4.2%, tricycle 3.7%, bicycle 3.6%, truck 2.8%, and bus 0.9%. Val does not repeat that order: person is 44.1% and car is 27.9%, with motor 10.5%, van 6.0%, bicycle 5.3%, tricycle 4.8%, truck 1.2%, and bus 0.2%. Bus stays under one percent in both splits. This notebook does not rebalance sampling, so a metric that averages the eight classes will move with bus.

The stacked bars are not flat across the training videos. The cell prints each video's leading class, the highest and lowest shares, every sequence whose boxes are a single class, any sequence with no object boxes, and any sequence whose object rows have no known class. On the last run the leading-class share ran from 34.6% to 100%. One video is a poor sample of the label set.

Label rows. The load cell counts short lines, non-positive width or height, coordinates outside `[0, 1]` by more than 0.001, and object rows with a missing or unknown class. A row can increment more than one counter. Short lines are not stored. Blank lines are ignored and not counted. The other flagged rows stay in the tables. Object rows with a missing or unknown class are left out of the COCO file because a COCO category has to exist.

Resolutions. The resolution cell prints the frame count at each native size and how many sequences contain more than one size. Native medians pool those sizes. The printed set is not one resolution: a box of the same pixel width on a short frame and on a frame about twice as wide is a different object after letterbox.

Object size, native pixels. The summarize cell prints the pooled native size. On the last run the median train box was 44×48 px (10th–90th percentile 19–126 px wide and 25–97 px tall), with area bins 24.9% small, 63.5% medium, and 11.6% large. Val was 47×57 px, with 19.2% / 69.5% / 11.3%. Native aspect p10 / p50 / p90 was 0.44 / 0.95 / 2.13. These figures are not the network input. If the resolution cell reports a mixed sequence, the new native print replaces them.

Object size at the input. Letterbox to a square of side S multiplies every box by `S / long_side`, so a side of 640 is exactly half the linear size of a side of 1280. The area thresholds stay fixed, so the small / medium / large shares are not halved with the side. The letterbox cell prints median width, height, p10, p90, and those bins at 1280 and at 640, then the same 1280 size by native resolution and by class. The per-class table is median width, height, and aspect. Anchors are not fit in this notebook.

Objects per frame. On the last run, train frames held a median of 43 objects (mean 45.7, max 180) and val a median of 37 (mean 40.1, max 116). The summarize cell names the sequence and the frame of the train maximum. The densest and sparsest sequences are the head and tail of that table. On the last run the busiest video was `uav0000138_00000_v`. The sparse tail is part of the same table: that is where median box area is larger, and the cell prints the Spearman correlation between mean objects per frame and median box area.

Ignore and empty frames. The coverage cell prints ignore boxes, how many frames contain one, frames with no object box, and frames with no box at all. Ignore regions go into the COCO file as `iscrowd=1` under a category named `ignore`. They are not training targets.

Neighboring frames. The lag cell prints, for train and for val, how many box pairs match at IoU 0.5 between consecutive stored frames and between frames five apart. Pairs with no object boxes on either frame are skipped. `FRAME_STRIDE` is that measurement. The loader does not drop frames.

Test. Test frames have no labels and are not in the COCO files. The summarize cell prints the clip count, frames per clip, and how many test frames sit at each resolution.

COCO files. `data/annotations/instances_train.json` and `instances_val.json` store each box in the original frame as `[x, y, width, height]`, with `area = width * height`. `pycocotools` prints category counts and its small / medium / large ranges on that native area. Those ranges use strict bounds. They describe the files. They are not the 1280 bins.

Choices that follow from this

Input resolution. Training letterboxes to **1280**. The comparison with 640 is the letterbox cell, because the shrink is uniform and the absolute size still depends on the native long side.

Geometry. Use the per-class medians at 1280 when a later model needs a size prior. This notebook does not emit anchors.

Augmentation. Horizontal flip has probability 0.5. Scale is drawn from `[0.5, 1.5]`, then the frame is cropped back to its original size or padded with 114, and only then letterboxed. The check prints mean object width at factors 0.5, 1, and 1.5 on one train frame. Train and validation stay split by sequence. The overlap cell prints how many sequence names are shared.

Preprocessing

Labels are YOLO text, one row per box: `class cx cy w h` with coordinates in `[0, 1]`. The same rows are written to COCO JSON in original-frame pixels during data understanding. The loader used here converts each row to `cls, x1, y1, x2, y2` in pixels. A row with no class is an ignore region and is stored as class `-1`. Ignore boxes are `iscrowd=1` in the JSON, they are drawn in yellow on the check, and they are not training targets. Pass objects and ignore regions through the same call if both should move together. The eight classes in `data.yaml` are the only targets.

Train and validation are the sequence folders as provided. The next cell prints how many sequence names are shared. Frames from one video are highly correlated, so a frame-level split would leak the scene into the score. Test clips are not used for training and are not in the COCO files.

`FRAME_STRIDE` is 5 and it is not applied. The lag cell measures consecutive stored frames and frames five apart. It does not remove frames from the loader.

Every split is letterboxed to 1280×1280. The aspect ratio stays as it is, and the empty border is filled with 114. The same scale and padding are applied to the boxes. Pixels then go through one `MinMaxScaler` fit on `[0, 255]`, which divides every pixel by 255. The scaler is not fit on the images. Train, validation, and test share this resize and this scaler.

Train-only augmentation uses `check_random_state`. Horizontal flip has probability 0.5. The frame is then resized by a factor in `[0.5, 1.5]` and fitted back to its original height and width: a larger frame is cropped at a random offset, a smaller frame is centered and padded with 114. Letterbox runs after that fit, so the factor is still present in the 1280 canvas. Validation and test are letterbox and the scaler only.

The figure shows one train frame, one val frame, and one test frame. The train row adds the same frame after a center crop at factor 1.5. The print under the figure is mean object width at 1280 for factors 0.5, 1, and 1.5. Red boxes are labeled objects. The test frame has no labels.

```
train_seq = {p.name for p in (ROOT / "train/images").iterdir() if p.is_dir()}
val_seq = {p.name for p in (ROOT / "val/images").iterdir() if p.is_dir()}
print("train sequences", len(train_seq))
print("val sequences", len(val_seq))
print("shared sequences", len(train_seq & val_seq))
```

```
train sequences 56
val sequences 7
shared sequences 0
```

Labels

```
# IMGSZ, PAD_VALUE, FRAME_STRIDE, and SEED are set with the imports.
# FRAME_STRIDE is only the lag measured above. This loader keeps every frame.
```

```
pixel_scaler = MinMaxScaler(feature_range=(0.0, 1.0))
pixel_scaler.fit(np.array([[0.0], [255.0]]))


def yolo_to_xyxy(label_path, width, height):
    # cls, x1, y1, x2, y2. A line with no class is an ignore region, stored as -1.
    rows = []
    label_path = Path(label_path)
    if not label_path.exists():
        return np.zeros((0, 5))
    for line in label_path.read_text().splitlines():
        vals = line.split()
        if len(vals) < 4:
            continue
        if len(vals) == 4:
            cls, cx, cy, bw, bh = -1.0, *map(float, vals)
        else:
            cls, cx, cy, bw, bh = map(float, vals[:5])
        rows.append([
            cls,
            (cx - bw / 2) * width, (cy - bh / 2) * height,
            (cx + bw / 2) * width, (cy + bh / 2) * height,
        ])
    return np.asarray(rows, dtype=np.float64) if rows else np.zeros((0, 5))
```

Letterbox

```
def letterbox_params(width, height, new_shape=IMGSZ):
    ratio = min(new_shape / height, new_shape / width)
    resized_w = int(round(width * ratio))
    resized_h = int(round(height * ratio))
    left = (new_shape - resized_w) // 2
    top = (new_shape - resized_h) // 2
    return ratio, left, top


def letterbox(image, boxes, new_shape=IMGSZ):
    height, width = image.shape[:2]
    ratio, left, top = letterbox_params(width, height, new_shape)
    resized = cv2.resize(image, (int(round(width * ratio)), int(round(height * ratio))))
    canvas = np.full((new_shape, new_shape, 3), PAD_VALUE, dtype=np.uint8)
    top = (new_shape - resized.shape[0]) // 2
    left = (new_shape - resized.shape[1]) // 2
    canvas[top:top + resized.shape[0], left:left + resized.shape[1]] = resized
    out = boxes.copy()
    if len(out):
        out[:, [1, 3]] = out[:, [1, 3]] * ratio + left
        out[:, [2, 4]] = out[:, [2, 4]] * ratio + top
    return canvas, out


def normalize(image):
    return image.astype(np.float32) / 255.0


def preprocess_val(image, boxes):
    boxed, boxed_boxes = letterbox(image, boxes)
    return normalize(boxed), boxed_boxes
```

Augmentation

```
def clip_boxes(boxes, width, height):
    if len(boxes) == 0:
        return boxes
    out = boxes.copy()
    out[:, 1] = np.clip(out[:, 1], 0, width)
    out[:, 3] = np.clip(out[:, 3], 0, width)
    out[:, 2] = np.clip(out[:, 2], 0, height)
    out[:, 4] = np.clip(out[:, 4], 0, height)
    keep = (out[:, 3] - out[:, 1] > 0) & (out[:, 4] - out[:, 2] > 0)
    return out[keep]
```

```
def scale_into_frame(image, boxes, factor, rng):
    # Fit back to the original frame so the later letterbox does not cancel the scale.
    height, width = image.shape[:2]
    scaled_w = max(1, int(round(width * factor)))
    scaled_h = max(1, int(round(height * factor)))
    scaled = cv2.resize(image, (scaled_w, scaled_h))
    out = boxes.copy()
    if len(out):
        out[:, 1:5] *= factor
    if scaled_w > width:
        left_src = int(rng.randint(0, scaled_w - width + 1))
    else:
        left_src = 0
    if scaled_h > height:
        top_src = int(rng.randint(0, scaled_h - height + 1))
    else:
        top_src = 0
    left_dst = 0 if scaled_w >= width else (width - scaled_w) // 2
    top_dst = 0 if scaled_h >= height else (height - scaled_h) // 2
    copy_w = min(scaled_w - left_src, width - left_dst)
    copy_h = min(scaled_h - top_src, height - top_dst)
    canvas = np.full((height, width, 3), PAD_VALUE, dtype=image.dtype)
    canvas[top_dst:top_dst + copy_h, left_dst:left_dst + copy_w] = scaled[
        top_src:top_src + copy_h, left_src:left_src + copy_w
    ]
    if len(out):
        out[:, [1, 3]] += left_dst - left_src
        out[:, [2, 4]] += top_dst - top_src
    return canvas, clip_boxes(out, width, height)


def preprocess_train(image, boxes, rng):
    boxes = boxes.copy()
    if rng.rand() < 0.5:
        image = np.ascontiguousarray(np.fliplr(image))
        if len(boxes):
            x1 = image.shape[1] - boxes[:, 3]
            x2 = image.shape[1] - boxes[:, 1]
            boxes[:, 1] = np.minimum(x1, x2)
            boxes[:, 3] = np.maximum(x1, x2)
    factor = float(rng.uniform(0.5, 1.5))
    image, boxes = scale_into_frame(image, boxes, factor, rng)
    return preprocess_val(image, boxes)
```

Check

```
def paint(image, boxes, color):
    view = image.copy()
    for row in boxes:
        x1, y1, x2, y2 = row[1:].astype(int)
        cv2.rectangle(view, (x1, y1), (x2, y2), color, 2)
    return view


def stack_boxes(objects, ignored):
    parts = [part for part in (objects, ignored) if len(part)]
    if not parts:
        return np.zeros((0, 5))
    return np.vstack(parts)


def by_kind(boxes):
    empty = np.zeros((0, 5))
    if len(boxes) == 0:
        return empty, empty
    ignore = boxes[:, 0] < 0
    return boxes[~ignore], boxes[ignore]


class CenterCrop:
    # The training crop is random. The check uses the center so the factor is visible.
    def randint(self, low, high):
```

```
        return low + (high - low - 1) // 2


def fit_letterbox(image, objects, ignored, factor, rng):
    fitted, fitted_boxes = scale_into_frame(
        image, stack_boxes(objects, ignored), factor, rng,
    )
    canvas, boxed = letterbox(fitted, fitted_boxes)
    boxed_objects, boxed_ignored = by_kind(boxed)
    return canvas, boxed_objects, boxed_ignored


train_paths = sorted((ROOT / "train/images/uav0000138_00000_v").glob("*.jpg"))
val_dir = sorted(p for p in (ROOT / "val/images").iterdir() if p.is_dir())[0]
val_paths = sorted(val_dir.glob("*.jpg"))
test_path = Path(test_df["image_path"].iloc[len(test_df) // 2])


def load_frame(split, sequence, stem):
    image = cv2.cvtColor(
        cv2.imread(str(ROOT / split / "images" / sequence / f"{stem}.jpg")),
        cv2.COLOR_BGR2RGB,
    )
    height, width = image.shape[:2]
    objects = yolo_to_xyxy(ROOT / split / "labels" / sequence / f"{stem}.txt", width, height)
    ignored = yolo_to_xyxy(ROOT / split / "ignore" / sequence / f"{stem}.txt", width, height)
    return image, objects, ignored


panels = [
    (*load_frame("train", "uav0000138_00000_v", train_paths[len(train_paths) // 2].stem), "train", True),
    (*load_frame("val", val_dir.name, val_paths[len(val_paths) // 2].stem), f"val/{val_dir.name}", False),
]
test_image = cv2.cvtColor(cv2.imread(str(test_path)), cv2.COLOR_BGR2RGB)
panels.append((test_image, np.zeros((0, 5)), np.zeros((0, 5)), f"test/{test_path.parent.name}", False))

fig, axes = plt.subplots(len(panels), 3, figsize=(14, 4 * len(panels)))
for row, (image, objects, ignored, title, show_scale) in zip(axes, panels):
    boxed, boxed_objects = letterbox(image, objects)
    _, boxed_ignored = letterbox(image, ignored)
    row[0].imshow(paint(paint(image, objects, (255, 40, 40)), ignored, (255, 220, 0)))
    row[0].set_title(f"{title}  objects={len(objects)}")
    row[0].axis("off")
    row[1].imshow(paint(paint(boxed, boxed_objects, (255, 40, 40)), boxed_ignored, (255, 220, 0)))
    row[1].set_title(f"letterbox {IMGSZ}")
    row[1].axis("off")
    norm = normalize(boxed)
    print(title, "letterbox normalized", float(norm.min()), float(norm.max()))
    if show_scale:
        zoom, zoom_objects, zoom_ignored = fit_letterbox(image, objects, ignored, 1.5, CenterCrop())
        row[2].imshow(paint(paint(zoom, zoom_objects, (255, 40, 40)), zoom_ignored, (255, 220, 0)))
        row[2].set_title("center crop x1.5, then letterbox")
        row[2].axis("off")
        print("mean object width at 1280, center crop")
        for factor in (0.5, 1.0, 1.5):
            _, fact_objects, _ = fit_letterbox(image, objects, ignored, factor, CenterCrop())
            if len(fact_objects) == 0:
                print(f"  factor {factor:.1f}: n=0")
                continue
            width = float((fact_objects[:, 3] - fact_objects[:, 1]).mean())
            print(f"  factor {factor:.1f}: n={len(fact_objects)} mean width {width:.1f}")
        rng = check_random_state(SEED)
        aug, aug_boxes = preprocess_train(image, stack_boxes(objects, ignored), rng)
        print(
            "preprocess_train normalized",
            float(aug.min()), float(aug.max()), "boxes", len(aug_boxes),
        )
    else:
        row[2].axis("off")
fig.tight_layout()
plt.show()
print("red = labeled objects, yellow = ignore. Test has no boxes.")
print("Training scale uses a random crop. The x1.5 panel is a center crop so the factor is easy to see.")
print("Edge boxes can be clipped, so the mean width is not exactly the factor.")
```

```
train letterbox normalized 0.0 1.0
mean object width at 1280, center crop
  factor 0.5: n=150 mean width 16.6
  factor 1.0: n=150 mean width 33.1
  factor 1.5: n=111 mean width 45.6
preprocess_train normalized 0.0 1.0 boxes 134
val/uav0000086_00000_v letterbox normalized 0.0 1.0
test/clip_025 letterbox normalized 0.0 1.0
```




```
red = labeled objects, yellow = ignore. Test has no boxes.
Training scale uses a random crop. The x1.5 panel is a center crop so the factor is easy to see.
Edge boxes can be clipped, so the mean width is not exactly the factor.
```

Write results to file

```
def run_dir(section, name):
    path = RESULTS / section / name
    path.mkdir(parents=True, exist_ok=True)
    return path

def read_json(path):
    if not path.exists():
        return None
    return json.loads(path.read_text())

def write_json(path, payload):
    path.write_text(json.dumps(payload, indent=2))

def same_protocol(path, protocol):
    saved = read_json(path)
    return saved is not None and saved.get("protocol") == protocol

def per_class_ap(evaluator):
    out = {names[int(class_id)]: 0.0 for class_id in sorted(names)}
    precision = None if evaluator.eval is None else evaluator.eval.get("precision")
    if precision is None:
        return out
    for index, cat_id in enumerate(evaluator.params.catIds):
        cat_id = int(cat_id)
        if cat_id not in names:
            continue
        vals = precision[:, :, index, 0, -1]
        vals = vals[vals > -1]
        out[names[cat_id]] = float(vals.mean()) if vals.size else 0.0
    return out

def save_history(section, name, protocol, history):
    write_json(run_dir(section, name) / "history.json", {
        "model": name,
        "protocol": protocol,
        "epochs": history,
    })
```

Eval

Every family is scored here, on the original frame. `VAL_COCO` is `instances_val.json` with each ignore region copied onto the eight object classes as `iscrowd=1`. The JSON file is not rewritten. Boxes predicted on the 1280 letterbox are mapped back before COCO AP, AP50, and AP_small.

```
from pycocotools.cocoeval import COCOeval


def scoring_coco(path):
    # Crowd boxes only excuse a detection of the same category. The file keeps ignore
    # under its own category, so each region is copied onto the eight object classes in memory.
    source = COCO(str(path))
    object_ids = [int(key) for key in sorted(names)]
    ignore_ids = {
        int(cat["id"])
        for cat in source.dataset["categories"]
        if cat.get("name") == "ignore"
    }
    dataset = {
        "images": source.dataset["images"],
        "categories": source.dataset["categories"],
        "annotations": list(source.dataset["annotations"]),
    }
    next_id = 1 + max((int(ann["id"]) for ann in dataset["annotations"]), default=0)
    attached = 0
    for ann in source.dataset["annotations"]:
        if int(ann["category_id"]) not in ignore_ids:
            continue
        for category_id in object_ids:
            cloned = dict(ann)
            cloned["id"] = next_id
            next_id += 1
            cloned["category_id"] = int(category_id)
```

```
                cloned["iscrowd"] = 1
                dataset["annotations"].append(cloned)
            attached += 1
        scoring = COCO()
        scoring.dataset = dataset
        scoring.createIndex()
        print(f"{Path(path).name}: {attached} ignore regions attached to {len(object_ids)} classes")
        if attached == 0:
            print("warning: no ignore category in", path)
        return scoring


    def letterbox_xyxy_to_original(boxes, width, height):
        ratio, left, top = letterbox_params(int(width), int(height), IMGSZ)
        out = np.array(boxes, dtype=np.float32, copy=True).reshape(-1, 4)
        out[:, [0, 2]] -= left
        out[:, [1, 3]] -= top
        out /= float(ratio)
        out[:, [0, 2]] = np.clip(out[:, [0, 2]], 0, width)
        out[:, [1, 3]] = np.clip(out[:, [1, 3]], 0, height)
        return out


    def coco_score_predictions(coco, predictions, image_ids):
        object_ids = [int(key) for key in sorted(names)]
        known = set(object_ids)
        allowed = set(int(image_id) for image_id in image_ids)
        per_class = {names[class_id]: 0.0 for class_id in object_ids}
        empty = {
            "mAP": 0.0,
            "AP50": 0.0,
            "AP_small": 0.0,
            "AP_per_class": per_class,
            "predictions": [],
        }
        cleaned = []
        for row in predictions:
            category_id = int(row["category_id"])
            image_id = int(row["image_id"])
            width, height = float(row["bbox"][2]), float(row["bbox"][3])
            score = float(row["score"])
            if category_id not in known or image_id not in allowed or width <= 0 or height <= 0 or score != score:
                continue
            cleaned.append({
                "image_id": image_id,
                "category_id": category_id,
                "bbox": [float(value) for value in row["bbox"]],
                "score": score,
            })
        if not cleaned:
            print("no predictions; AP is 0")
            return empty
        detected = coco.loadRes(cleaned)
        evaluator = COCOeval(coco, detected, "bbox")
        evaluator.params.catIds = object_ids
        evaluator.params.imgIds = sorted(allowed)
        evaluator.evaluate()
        evaluator.accumulate()
        evaluator.summarize()

        def clean(value):
            value = float(value)
            if value != value or value < 0:
                return 0.0
            return value

        return {
            "mAP": clean(evaluator.stats[0]),
            "AP50": clean(evaluator.stats[1]),
            "AP_small": clean(evaluator.stats[3]),
            "AP_per_class": per_class_ap(evaluator),
            "predictions": cleaned,
        }


    def eval_frame_path(dataset, index):
        if hasattr(dataset, "paths"):
```

```
            return dataset.paths[index]
        return dataset.base.paths[dataset.indexes[index]]


    def coco_score(model, dataset):
        model.eval()
        index = {}
        sizes = {}
        for image in VAL_COCO.dataset["images"]:
            image_id = int(image["id"])
            index[(image["sequence"], str(image["frame"]))] = image_id
            sizes[image_id] = (int(image["width"]), int(image["height"]))
        detections = []
        image_ids = []
        for item in range(len(dataset)):
            path = Path(eval_frame_path(dataset, item))
            key = (path.parent.name, str(path.stem))
            if key not in index:
                raise KeyError(f"{path} is not in the validation COCO file")
            image_id = index[key]
            image_ids.append(image_id)
            image, _ = dataset[item]
            with torch.no_grad():
                pred = model([image.to(device)])[0]
            boxes = pred["boxes"].detach().cpu().numpy()
            labels = pred["labels"].detach().cpu().numpy()
            scores = pred["scores"].detach().cpu().numpy()
            width, height = sizes[image_id]
            if len(boxes):
                boxes = letterbox_xyxy_to_original(boxes, width, height)
            for box, label, score in zip(boxes, labels, scores):
                category_id = int(label) - 1
                if category_id not in names:
                    continue
                x1, y1, x2, y2 = (float(value) for value in box)
                if x2 <= x1 or y2 <= y1 or float(score) != float(score):
                    continue
                detections.append({
                    "image_id": image_id,
                    "category_id": category_id,
                    "bbox": [x1, y1, x2 - x1, y2 - y1],
                    "score": float(score),
                })
        return coco_score_predictions(VAL_COCO, detections, image_ids)


    val_json = ROOT / "annotations" / "instances_val.json"
    if not val_json.is_file():
        raise FileNotFoundError(
            "instances_val.json is missing. Run the COCO cell above. This section does not rewrite it."
        )
    VAL_COCO = scoring_coco(val_json)

    def milliseconds_per_frame(model, n=100):
        model.eval()
        frames = [val_set[i][0].to(device) for i in range(n)]
        for image in frames[:10]:
            with torch.no_grad():
                model([image])
        torch.cuda.synchronize()
        start = time.perf_counter()
        for image in frames:
            with torch.no_grad():
                model([image])
        torch.cuda.synchronize()
        return 1000 * (time.perf_counter() - start) / n
```

```
loading annotations into memory...
Done (t=0.14s)
creating index...
index created!
creating index...
index created!
instances_val.json: 3995 ignore regions attached to 8 classes
```

Two-stage Anchor-based

```
from torchvision.models.detection import (
    fasterrcnn_resnet50_fpn_v2,
    FasterRCNN_ResNet50_FPN_V2_Weights,
)
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
from pycocotools.coco import COCO
from pycocotools.cocoeval import COCOeval

import time
import torch
from torch.utils.data import Dataset, DataLoader
```

```
device = torch.device("cuda")
num_classes = len(names) + 1

class DroneDet(Dataset):
    def __init__(self, split, train=False):
        self.split = split
        self.train = train
        self.epoch = 0
        self.paths = []
        for seq_dir in sorted((ROOT / split / "images").iterdir()):
            if seq_dir.is_dir():
                self.paths += sorted(seq_dir.glob("*.jpg"))

    def __len__(self):
        return len(self.paths)

    def __getitem__(self, index):
        path = self.paths[index]
        image = cv2.cvtColor(cv2.imread(str(path)), cv2.COLOR_BGR2RGB)
        height, width = image.shape[:2]
        boxes = yolo_to_xyxy(
            ROOT / self.split / "labels" / path.parent.name / f"{path.stem}.txt",
            width, height,
        )
        if self.train:
            rng = check_random_state(SEED + self.epoch * len(self.paths) + index)
            image, boxes = preprocess_train(image, boxes, rng)
        else:
            image, boxes = preprocess_val(image, boxes)
        keep = boxes[:, 0] >= 0 if len(boxes) else np.zeros(0, dtype=bool)
        boxes = boxes[keep]
        image = torch.from_numpy(image).permute(2, 0, 1).contiguous()
        if len(boxes) == 0:
            target = {
                "boxes": torch.zeros((0, 4), dtype=torch.float32),
                "labels": torch.zeros((0,), dtype=torch.int64),
            }
        else:
            target = {
                "boxes": torch.tensor(boxes[:, 1:5], dtype=torch.float32),
                "labels": torch.tensor(boxes[:, 0] + 1, dtype=torch.int64),
            }
        target["image_id"] = torch.tensor([index])
        return image, target


def collate(batch):
    return tuple(zip(*batch))


train_loader = DataLoader(DroneDet("train", train=True), batch_size=2, shuffle=True,
                          num_workers=0, collate_fn=collate, pin_memory=True)

val_set = DroneDet("val", train=False)
val_loader = DataLoader(val_set, batch_size=2, shuffle=False, num_workers=0, collate_fn=collate)
```

```
def build_model():
    model = fasterrcnn_resnet50_fpn_v2(
        weights=FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT,
```

```
            min_size=IMGSZ, max_size=IMGSZ
        )
        in_features = model.roi_heads.box_predictor.cls_score.in_features
        model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
        return model.to(device)
```

```
    def fit_2_anchor(model, name, positive_fraction=None, schedule=False):
        if positive_fraction is not None:
            model.roi_heads.fg_bg_sampler.positive_fraction = positive_fraction
        fraction = float(model.roi_heads.fg_bg_sampler.positive_fraction)
        print("positive fraction", fraction)
        protocol = {
            "epochs": 5, "lr": 1e-4, "imgsz": IMGSZ,
            "optimizer": "AdamW", "images_per_step": 2,
            "positive_fraction": fraction, "schedule": schedule,
        }
        folder = run_dir("two_stage", name)
        weights_path = folder / "best.pt"
        if same_protocol(folder / "val.json", protocol):
            print("skip", name)
            if weights_path.is_file():
                model.load_state_dict(torch.load(weights_path, map_location=device))
            saved = read_json(folder / "history.json")
            model.history = saved["epochs"] if saved else []
            return model

        optimizer = torch.optim.AdamW(
            [p for p in model.parameters() if p.requires_grad],
            lr=1e-4, weight_decay=0.0005,
        )
        scheduler = (
            torch.optim.lr_scheduler.StepLR(optimizer, step_size=4, gamma=0.1)
            if schedule else None
        )

        n_images = len(train_loader.dataset)
        history = []
        best_map = -1.0
        best_epoch = None
        best_row = None
        for epoch in range(5):
            train_loader.dataset.epoch = epoch
            model.train()
            total = 0.0
            for images, targets in train_loader:
                images = [im.to(device) for im in images]
                targets = [{k: v.to(device) for k, v in t.items()} for t in targets]
                optimizer.zero_grad(set_to_none=True)
                loss = sum(model(images, targets).values())
                loss.backward()
                optimizer.step()
                total += float(loss)

            if scheduler is not None:
                scheduler.step()

            row = coco_score(model, val_set)
            history.append({
                "epoch": epoch + 1,
                "loss": total / len(train_loader),
                "images_seen": n_images * (epoch + 1),
                "mAP": row["mAP"], "AP50": row["AP50"], "AP_small": row["AP_small"],
            })
            save_history("two_stage", name, protocol, history)
            print(f"epoch {epoch + 1}  loss {history[-1]['loss']:.3f}  mAP {row['mAP']:.3f}")
            if row["mAP"] > best_map:
                best_map = row["mAP"]
                best_epoch = epoch + 1
                best_row = row
                torch.save(model.state_dict(), weights_path)

        model.load_state_dict(torch.load(weights_path, map_location=device))
        ms = milliseconds_per_frame(model)
        write_json(folder / "val.json", {
            "model": name,
            "protocol": protocol,
```

```
        "metrics": {
            "AP": best_row["mAP"], "AP50": best_row["AP50"], "AP_small": best_row["AP_small"],
            "AP_per_class": best_row["AP_per_class"],
            "ms_per_frame": ms, "images_seen": n_images * 5,
            "best_epoch": best_epoch,
        },
        "predictions": best_row["predictions"],
    })

    model.history = history
```

```
initial_2_anchor = fit_2_anchor(build_model(), "fasterrcnn")
```

```
positive fraction 0.25
Loading and preparing results...
DONE (t=0.39s)
creating index...
index created!
Running per image evaluation...
Evaluate annotation type *bbox*
DONE (t=28.64s).
Accumulating evaluation results...
DONE (t=1.05s).
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.215
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.409
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.199
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.061
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.183
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.410
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.110
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.256
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.312
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.124
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.271
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.527
epoch 1  loss 0.765  mAP 0.215
Loading and preparing results...
DONE (t=0.46s)
creating index...
index created!
Running per image evaluation...
Evaluate annotation type *bbox*
DONE (t=27.96s).
Accumulating evaluation results...
DONE (t=1.00s).
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.205
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.394
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.186
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.066
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.185
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.375
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.110
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.254
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.308
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.133
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.263
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.519
epoch 2  loss 0.642  mAP 0.205
Loading and preparing results...
DONE (t=0.51s)
creating index...
index created!
Running per image evaluation...
Evaluate annotation type *bbox*
DONE (t=26.05s).
Accumulating evaluation results...
DONE (t=1.00s).
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.192
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.362
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.179
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.073
```

```
tuned_2_anchor = fit_2_anchor(build_model(), "fasterrcnn_tuned", positive_fraction=0.5, schedule=True)
```

```
positive fraction 0.5
Loading and preparing results
```